

Pushing ROS towards the Dark Side: A ROS-based Co-Simulation Architecture for Mixed-Reality Test Systems for Autonomous Vehicles

Marc René Zofka¹, Lars Tötzel¹, Maximilian Zipfl¹, Marc Heinrich¹,
Tobias Fleck¹, Patrick Schulz² and J. Marius Zöllner¹

Abstract—Validation and verification of autonomous vehicles is still an unsolved problem. Although virtual approaches promise a cost efficient and reproducible solution, a most comprehensive and realistic representation of the real world traffic domain is required in order to make valuable statements about the performance of a highly automated driving (HAD) function. Models from different domain experts offer a repository of such representations. However, these models must be linked together for an extensive and uniform mapping of real world traffic domain for HAD performance assessment.

Hereby, we propose the concept of a co-simulation architecture built upon the Robot Operating System (ROS) for both coupling and for integration of different domain expert models, immersion and stimulation of real pedestrians as well as AD systems into a common test system. This enables a unified way of generating ground truth for the performance assessment of multi-sensorial AD systems. We demonstrate the applicability of the ROS powered co-simulation by coupling behavior models in our mixed reality environment.

I. INTRODUCTION

The validation and verification (V&V) of highly automated and autonomous vehicles is still an unsolved problem. Various methods of testing and validation are applied along the automotive development model, which is largely derived from the V-model [1]. Components and systems are gradually developed and tested step by step via formal validation, simulations, various types of test benches and finally real test drives. Step by step, one also loses the properties of observability and controllability. For highly automated driving (HAD), techniques are required to combine realism with expert models of different disciplines. For this purpose, software architectures on the one hand need to provide integratability of models for different situation aspects of a traffic scenario, but also an interchangeability of models of the same aspect. Furthermore, the continuity of the development process, respectively the application of scenarios and tooling, shall be achieved, allowing components being successively exchanged from virtual to real counterparts.

Current research shows that the linking of simulation models enables the exploration of new findings in the field of HAD, such as human factors or vulnerable road user

¹Authors are with Department of Technical Cognitive Assistance Systems, FZI Research Center for Information Technology, 76131 Karlsruhe, Germany {zofka, toetzel, zipfl, heinrich, tfleck, zoellner}@fzi.de

²Author is with the Department of Microelectronic System Design (SiM), FZI Research Center for Information Technology, 76131 Karlsruhe, Germany pschulz@fzi.de



Fig. 1: The ROS-based co-simulation architecture powers the virtual reality cave, linking real physical components and virtual domain expert models into a common traffic scenario.

(VRU) safety. To provide an efficient and effective coupling, concepts and frameworks are therefore required to link models for individual aspects of traffic situations. Many approaches are currently based on a bilateral coupling of two models, furthermore, the integration of multiple models offers a comprehensive realism resulting from different domain expertises. Especially from an academic point of view, concepts and frameworks are needed that allow a fast implementation of these couplings. Open source frameworks might facilitate the development and linking of these models. Thus, this paper proposes a methodology for mapping and linking simulation models using the *Robot Operating System* (ROS) [2] at the example of different road user behavior. Where most use ROS for AD prototypes, we hereby propose an approach to use ROS as the core of a flexible co-simulation architecture to challenge the AD systems. Therefore, a mixed reality test system has been built up upon our former work [3] and is powered by the hereby presented ROS-based co-simulation.

This paper is structured as follows: In Sec. II we give a brief overview of the related work in the field of validation and verification of automated driving, with an emphasis on test system architectures. In Sec. III we give an in-depth introduction into our abstract architecture concept, followed by mapping instructions to ROS in Sec. IV. We evaluate our approach in Sec. V, and finally give a summary and outlook in Sec. VI.

II. RELATED WORK

Several functional system architectures are known from the field of academic research, see the overview on automated vehicles [4]. Functional system architectures are presented in [5], [6], [7], for example. For a detailed consideration and differentiation of different functional system architectures, we refer to the given in overview [8].

Accompanying the development, components and systems derived from these architectures are to be tested. Therefore a lot of different simulation frameworks exist: Commercial tools, e.g. *VIRES Virtual Test Drive* (VTD) or *IPG Car-Maker*, as well as academic and community-driven open source approaches, such as *CARLA* [9], enable the development and testing of HAD. Each of them focuses on special applications, such as the AI based approach *Reinforcement Learning*, the mapping of realistic vehicle dynamics or continuity across different test system instances along the *X-in-the-loop* paradigm. For a detailed comparison of different simulation frameworks see [10] and [11].

Most approaches are based on a bilateral coupling of two models. In doing so, some approaches focus on a linkage between traffic and vehicle dynamics simulation, such as [12]. In our former work [13] we demonstrated the coupling of a real experimental vehicle by *Vehicle-in-the-loop* (ViL) with *DLR SUMO* [14] by the use of *GAZEBO* [15] and a ROS-based communication. Other work focuses on using ROS as a interface to *Unity3D* [16] for the simulation environment *3DCoAutoSim* [17], [18]. In [19] a bilateral coupling between VTD, *Automotive Data and Time-Triggered Framework* (ADTF) and the pedestrian simulation *MomentUM* [20] has been presented.

ROS [2] serves as a operating system for distributed systems in robots, mobile systems and automated vehicles. As shown in [21], ROS is used primarily as an interface rather than a test systems foundation. So, some simulations offer interfaces or plugins for integration, such as *Gazebo* [15] or *CARLA* [9]. For example, in the multi-vehicle simulation framework *CoInCar-Sim* [22], ROS is used to integrate automated vehicle algorithms.

Some approaches try to unify the linking and communication between different models of a simulation in order to facilitate the exchangeability of models and analysis modules. An early approach of a networking and communication protocol is the IEEE Standard for *Distributed Interactive Simulation* (DIS) [23], [24]. Current approaches pursue *Functional Mockup Units* (FMU) [25] and the *Distributed Co-Simulation Protocol* (DSP) [26]. Even a coupling for FMUs and ROS has been presented in [27]. Currently, also the *Open Simulation Interface* [28] provides generic messages for HAD testing purposes.

To the best of the author's knowledge, no approach exists in order to use ROS as a underlying co-simulation architecture to realize a simulation framework, which enables the integration of arbitrary behavior models for the different traffic participants, integrating real entities as well as virtual models. This is the starting point of the present paper.

III. CONCEPT OF THE SOFTWARE ARCHITECTURE

A. Formalism and terminology

In the following we consider traffic scenarios according to the nomenclature of Ulbrich et al. [29] A temporal snapshot of a traffic scenario is called traffic scene. We formalize this as a compound state $\mathcal{U}_t$ of all relevant entities, such as vehicles V , pedestrians P , but also traffic lights T at a distinct time t . A separate variable is introduced for the *system-under-test* (SuT), resp. the vehicle ϵ integrating the HAD function:

$$\mathcal{U}_t := \{V_1, \dots, V_n, P_1, \dots, P_n, T_1, \dots, T_n, \epsilon\} \quad |\mathcal{U}_t| = m$$

An entity's state is represented by its three-dimensional position given by x, y and z , a three-dimensional rotation ψ, ϕ and δ and an additional state s , which models traffic participants specific attributes (signal states, gestures, ...).

$$\vec{x} = (x, y, z, \psi, \phi, \delta, s)$$

The temporal development of a traffic scenario $\mathcal{U}_t \rightarrow \mathcal{U}_{t+1}$ depends on the dynamics of the individual traffic participants and the behavior of the SuT ϵ . The dynamics of the individual road users are modeled by a global function $\Omega(t)$, which can be partitioned into several class-dependent models for vehicles $\Omega_V(t)$, for pedestrians $\Omega_P(t)$ as well as semi-dynamic states of the traffic lights, resp. signal phases and timing $\Omega_T(t)$.

B. Client-Server Architecture

Each of these class-dependent models can now be integrated within a client-multi-server network. Fig. 2 shows a simple client-server architecture.

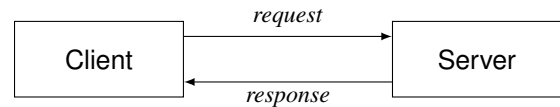


Fig. 2: Simple client-server principle: A client requests a service or data from a connected server.

Servers representing such a model provide the functions necessary for parameterization and time propagation. Therefore, every model is conceptualized as a server component, that offers various services, such as propagating towards a certain simulation time. The *simulation kernel* is realized as a client component, which administrates the connected model.

By integrating multiple models, this approach results in a client-multi-server approach, where the simulation kernel handles all the different, connected models and administrates the linkage and data exchange between them, see Fig. 3. This involves an a-priori step, the parametrization of the connected models as well as handling them at simulation runtime. The simulation kernel therefore embodies a state machine with the following states:

- 1) *Initialization*: A selected scenario is traced back to several parameters, i.e. configuration files that configure the different connected models.

- 2) **Simulation:** The simulation kernel loops iteratively by a given signal from $t = 0$ with Δt . If a defined criterion is fulfilled or time constraints cannot be held by the connected models, it ends the loop.
- 3) **Post-processing:** In the post-processing phase, corresponding quantities are determined, metrics for the performance assessment are evaluated and the data is stored.

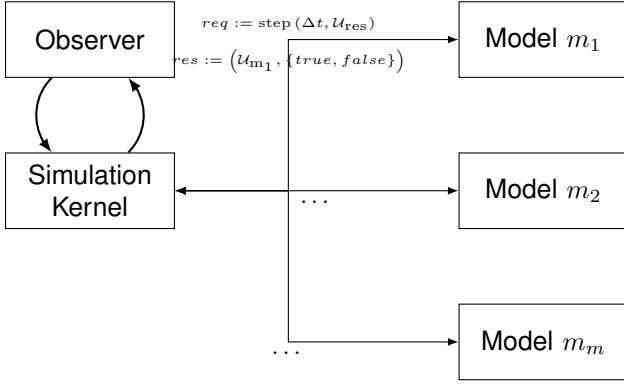


Fig. 3: Client-Multi-Server approach: The client, resp. simulation kernel, triggers different servers, each representing a class-specific behavior model. The request asks for time progression within the model by Δt with a residual environmental model $\mathcal{U}_{res}$.

The behavior models are to be realized as servers, providing services to a calling client, such as propagating the time in the internal behavior and scene graph model. Typically, behavior models do not only provide a logical behavior model, but also some kind of internal data structure for a scene graph model. This is especially the case, if perceptual aspects of the modeled entities are reflected in some entities' behavior. Therefore, a matching of the global with the local environment model takes place.

A behaviour model must offer the following services:

- **init():** Initialization routine checks, whether all models are available.
- **configure():** The configure function enables the configuration of the scenario to be taken place in each model as well as the behavior's model parametrization.
- **step(Δt):** The step function propagates the model in time.
- **stop():** The models finally derive key performance indicators, but do not reset them.
- **reset():** The reset function internally resets all condition and measurement variables.

Observers in general, see Fig. 3, provide the capability to observe and access ground truth quantities in two ways: Firstly, we use observed measures to calculate metrics which provide a performance assessment of the SuT and to check constraints, which might cause a test abort. Secondly, observers are graphical user interfaces to integrate and inform about the current internal quantities of the running simulation.

IV. REALIZATION BY ROS

In the following we focus on the mapping and realization of the beforementioned architecture to ROS:

- **ROS model wrappers** integrate real world entities and respective models as distributed processes
- **ROS interprocess communication (IPC)** couples those models and real world entities and
- **ROS ecosystem** adapts visualizations and provides condition monitoring of the coupled models.

In order to integrate real entities and respective models in the ROS-powered simulation kernel, we propose a *tightly* and *loosely coupled model wrapper*. Therefore, two adapters are realized, see Fig. 4. Both are responsible for the harmonisation of coordinate systems, tool-specific and general datatypes as well as model-specific quantities' visualization.

Synchronous wrappers are used to embed behavior models that provide direct feedback on the success and failure and resulting behavior of the managed entities. Asynchronous wrappers cascade real experimental vehicles (ViL) and pedestrians integration, so called *avatars* [3], which do not provide an real-time response. Here, reliable service calls are converted into topics using buffer data structures, thus enabling time-shifted, asynchronous embodiment. Time constraints or position quality are checked continuously to provide a consistent integration. Besides coordinate systems' harmonization, the wrappers also handle the conversion globally unique to model-specific identifier.

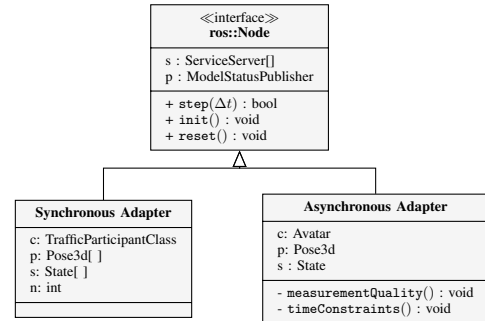


Fig. 4: UML class diagram: Synchronous and asynchronous wrapper enable either a tight or loose coupling of models. Asynchronous wrapper for real entities check the validity of the integrated experimental vehicle and the immersed pedestrian.

We introduce one dedicated ROS node, which represents the role of a simulation kernel and handles the communication and data exchange using ROS *interprocess communication* by means of topics and services. In Listing 1 the service **step()** is illustrated. The partial environment model is given as a part of the service's request. So, each wrapper is aware of the externally handled and controlled traffic participant entities. Beside the service definitions, multiple topics are defined for the partial environment models, also facilitating a uniform visualization approach, using tools of the ROS *ecosystem*, such as *rqt* or *rviz*.

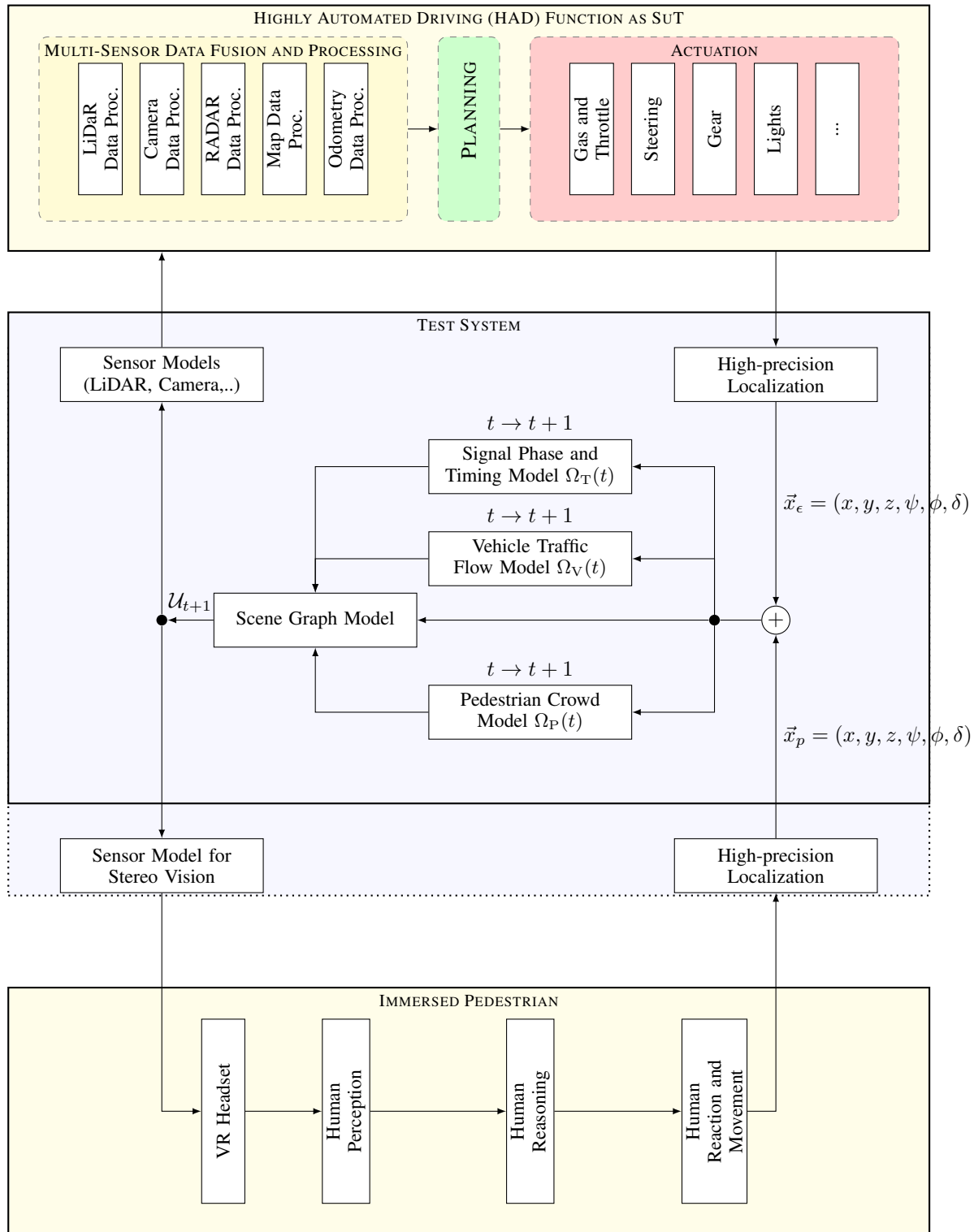


Fig. 5: Block diagram illustration of the test system and the SuT in a common test loop: The HAD function as SuT receives data generated as stimuli from virtual sensor models and determines an vehicle's reaction based on this input (upper section). The vehicle's reaction, e.g. in terms of its change of position over time is in turn passed on to the test system (middle section). Here the co-simulation takes this information and passes it towards the connected models for temporal situation propagation. The same applies to the immersed pedestrian (lower section): The pedestrian receives a rendered representation of the global scene graph model via VR headset. He reacts according to his own perception and reasoning. His change of position is observed by an external reference localization system and is also forwarded to the global environment model.

Listing 1: ROS service definition of `step()`

```
float32 target_time # Target time
TrafficLightStateArray traffic_light_states # Residual traffic light states
fzi_sensor_msgs/ObstacleArray traffic_participants # Residual world model

bool success # indicates successful time propagation
string message # informational, e.g. for error messages
float32 sim_time # Simulation time, up to which the world model was updated.
TrafficLightStateArray traffic_light_states # Updated partial world model
fzi_sensor_msgs/ObstacleArray traffic_participants # Updated partial world model
```

The co-simulation architecture has been realized using different frameworks, which embody domain expert models for different traffic participant behaviors: *DLR SUMO* [14] for vehicle flow and *UNC Menge* [30] for pedestrian crowd simulation. In the following, their integration by means of the co-simulation is described in detail. Fig. 5 and Fig. 7 illustrate the integration of the co-simulation with an exemplary SuT and the sequential control via the simulation kernel.

A. SUMO as Traffic Flow Simulation

In our co-simulation architecture we use SUMO as an exemplary integration to instantiate the vehicle flow behavior $\Omega_V(t)$ to gain advantage of the variety of car-following models integrated in SUMO. Our SUMO wrapper uses the *Traffic Control Interface* (TraCI) [31], providing the ability to interact with SUMO using TCP. On top of TraCI, our wrapper makes use of the synchronous adapter, immediately providing feedback on the update of SUMO. Therefore, a *ScenarioManager* class is implemented, which provides the capability to change scenario parameters a priori and to step forward with scenarios. It also offers the proposed `step()`, `reset()` and `stop()` ROS services, mapping them to a suitable sequence of TraCI calls.

After a service request is received, the included residual environment model is passed to SUMO. The residual environment model includes all traffic participants and objects, that are not generated and managed by SUMO, such as (immersed) pedestrians or the SuT. Then, all vehicles controlled by SUMO are updated using TraCI's `step` method and finally, the updated environment model is fed back to the *simulation kernel node* via service response.

Additionally, all traffic signals and their signal phases $\Omega_T(t)$ within the scenario are controlled by SUMO. Consequently, the service response includes, besides the states of all vehicles, the traffic signal information. The visualization includes both the road network and the individual road users. This is especially useful for condition monitoring and debugging purposes.

B. Menge as Pedestrian Crowd Simulation

Pedestrian crowd behavior $\Omega_P(t)$ is integrated by utilizing *UNC Menge* Crowd Simulation Engine [30]. Menge provides a variety of pedestrian models and functions as an integration platform and benchmarking suite for developers of such models. Thus the integration of Menge provides valuable state of the art motion models. We implemented a ROS wrapper for Menge that allows to insert, remove and move externally controlled agents in Menge according to signals provided to a ROS interface. The state of simulated agents in Menge can be queried by the implemented interface as well. Time synchronization of Menge to the simulation kernel is

ensured by providing a ROS service interface `step()` that allows to update the time in Menge.

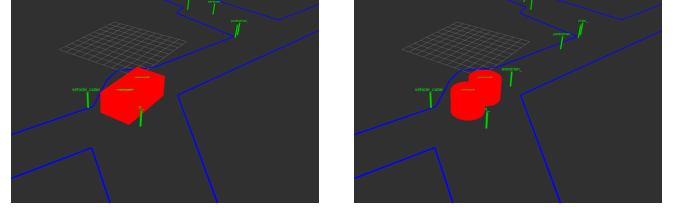


Fig. 6: Approximation of the simulated environment model in Menge: The left picture shows the true size of a vehicle that is inserted into the co-simulation, the right picture shows the corresponding approximation of the vehicle with two circular shapes in Menge. The walkable area is coloured blue.

Since Menge is constructed as a plain pedestrian behavior simulation that assumes dynamic agents to have circular shape, we extend the *Pedestrian Velocity Model* (PedVO) of [32] by approximating agents of other shapes and types, like driving vehicles, with multiple subagents connected as a rigid body. Fig. 6 shows the approximation of a vehicle by multiple circular subagents. Static obstacles of arbitrary polygon shape can be defined in Menge a priori and modeled directly for a given traffic scenario.

C. Unreal Engine as Scene Graph Model

The unified scene graph engine to provide the access point and interface for multi-sensor data is realized by using the *Unreal Engine* (UE). This enables the creation of realistic sensor data, such as optical camera and LiDAR sensor data out of UE and thereby the stimulation of an immersed pedestrian as well as the SuT likewise, see Fig. 5.

UE is connected to the *simulation kernel node*, which handles the exchange of traffic and pedestrian updates with SUMO and Menge and is responsible for converting binary data from and to an UE friendly format. While the models are distributed across different nodes, they are connected to a *rosbridge* [33], which allows access to all ROS functionalities by making topics and services available over sockets. The *rosbridge* integration is based on [34].

The integration is extended by adding the beforementioned topics and services as well as integrating new data types. Incoming messages contain the vehicles', pedestrians' and traffic lights' states $\vec{x}$, whereas outgoing messages include the location, orientation and velocity of any immersed pedestrian. An additional conversion step includes the coordinate systems' harmonization, resulting in flipping the systems y-axis and handling offsets, which are depending on the used map data and its geometric origin. By parsing all environment actors present in the scene and comparing them to the list of incoming traffic participants, either new actors are added to or out of date actors are removed from the scene, supporting the scene graph's sanity and improving overall performance as the message update rate is heavily dependent on the actual framerate of the engine.

The user can immerse into the traffic scene and interact with other models using a VR headset. In contrast to the

beforementioned models, the UE wrapper is coupled by an asynchronous adapter. This results from the fact that when using a VR headset, a synchronous step with the simulation kernel might result in a laggy visualization. The current poses of the immersed pedestrian and vehicle are stored in one buffer element and the latest partial environments of all other models are retrieved from another buffer of the asynchronous adapter. Consequently, UE is coupled loosely.

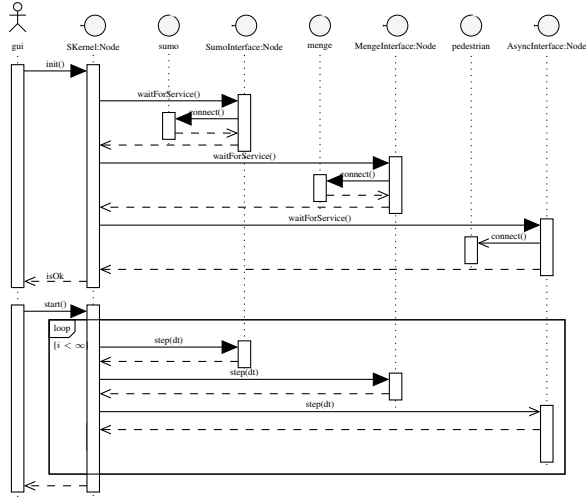


Fig. 7: UML Sequence Diagram: Sequential calls of the different models by the simulation kernel.

V. EVALUATION

We pursue the following approach to evaluation: In order to demonstrate the usefulness of combining the different models, we apply the co-simulation in a concrete use case. To evaluate co-simulation against other options, we discuss the properties of ROS against selected alternatives, see Tab. I.

TABLE I: Co-Simulation frameworks.

	Co-Simulation Frameworks		
	Functional Mockup Interface (FMI)	Distributed Co-Simulation Protocol (DSP)	Present approach based on ROS
Datatypes / Serialization	Scalars only (OSI: Arrays and complex datatypes via Protobuf)	Scalars, arrays and complex binary types (as PDU)	Scalars and arrays, compound messages
IPC	Shared memory concept	TCP/UDP over IP	TCP/UDP over IP
Integrated Tooling	cust. tools necess.	cust. tools necess.	rviz, rqt, rosbag
Native language support	C	none	C++, Python, Lisp, Java (messages)
comp. to proto. models	as FMUs	Adapters necessary	via ROS wrapper

We have built a virtual reality cave with inner extensions of about 5×4.5 metres, see Fig. 1. Based on the experiences of our former mixed-reality framework for pedestrians and vehicles [3], we have improved the setup by now using

IEEE 802.11ad based wireless HTC VIVE glasses. In total four Lighthouse tracking systems are attached to the traverse system in order to achieve a workspace of more than $25m^2$. The integration of roadside units, which communicate over WLAN 802.11p, enable the integration of real experimental vehicles if they are moved outside the laboratory.

To prove the functionality of the co-simulation architecture, a real section of the Test Area Autonomous Driving Baden Württemberg was mapped within the co-simulation. For this purpose, the different routes were modelled in UE, SUMO and in Menge. Due to the lack of standards, this was done using a highly accurate map in OpenDRIVE format, a manually post-modified map from OpenStreetMap data and a metric map for Menge. The following Fig. 9 illustrates a critical jaywalking scenario in which the vehicle flow and the pedestrian crowd simulation, an exemplary SuT as well as an immersed pedestrian interacting with each other are integrated.

VI. CONCLUSION AND OUTLOOK

We have proposed a ROS-based co-simulation architecture, which facilitates the integration of real and virtual traffic participants into a common mixed reality framework. The resulting architecture is based on a layer model, see Fig. 8. The resulting framework now enables the holistic simulation of traffic scenarios by coupling multiple domain expert models with real world entities, and using the advantages of ROS distributed system modeling approach, its IPC and ecosystem capabilities. This framework now enables an extensive V&V of HAD functions.

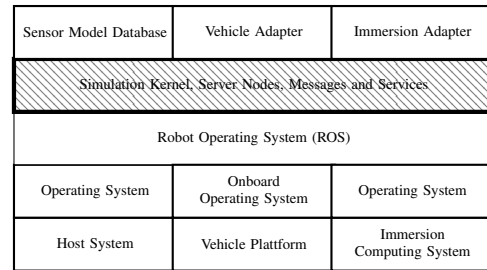
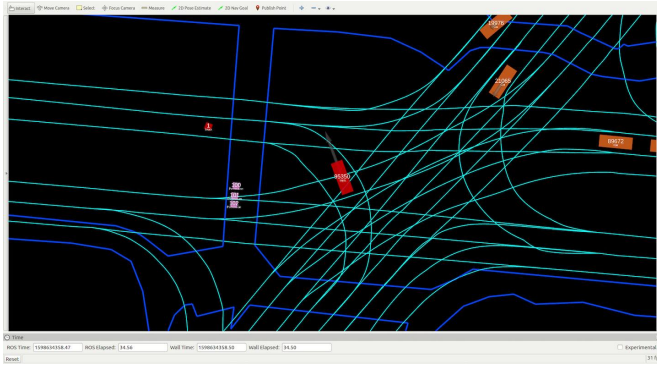


Fig. 8: Layer Model: Our proposed co-simulation architecture consists of ROS nodes, topic and service definitions as well as visualization messages, based on ROS' capabilities.

For future research work, we have identified several upcoming research questions: First, approaches for the sampling of the parameter space, which is opened via our co-simulation, must now be developed. Second, we want to determine the error propagation and the effects on the simulation results when localization systems are involved when immersing pedestrians and a real vehicle. Third and last, we would like to investigate the new capabilities of ROS2 for porting the co-simulation architecture. Our presented approach is also used in the project Drive2TheFuture for the modeling of human interaction behavior and derived from it optimizations of automated vehicles [35].



(a) The different environment models visualized in rviz at t_0 .



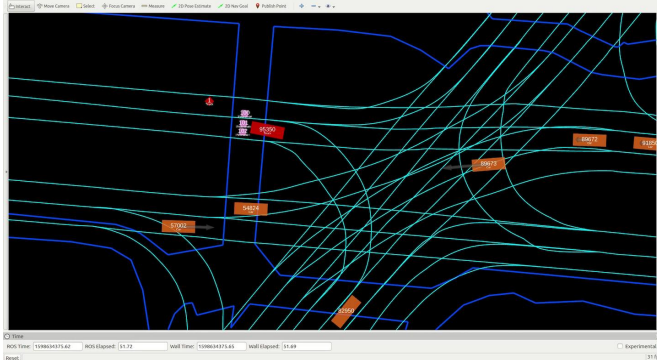
(b) Resulting global scene graph model in UE at t_0 .



(c) The different environment models visualized in rviz at t_1 .



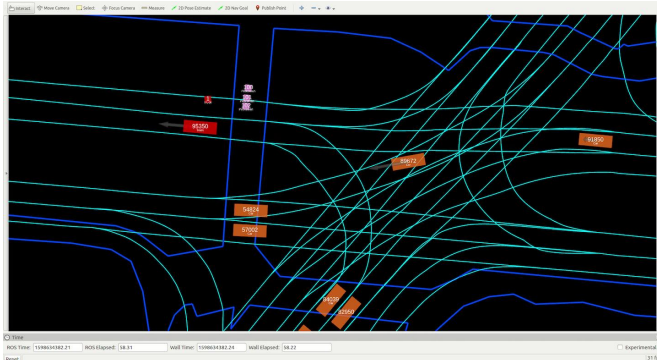
(d) Resulting global scene graph model in UE at t_1 .



(e) The different environment models visualized in rviz at t_2 .



(f) Resulting global scene graph model in UE at t_2 .



(g) The different environment models visualized in rviz at t_3 .



(h) Resulting global scene graph model in UE at t_3 .

Fig. 9: The co-simulation sequentially updates different models into a common scene graph model, visualized in rviz (left). Out of this, the scene graph model in UE feeds the sensor models (right) to stimulate the immersed pedestrian and the HAD function. SUMO controlled vehicles (orange) interact with the Menge pedestrians (light pink) and with the immersed pedestrian and an exemplary SuT (left: both red, right: both black). Model-specific environment models are visualized in rviz in a consistent way, such as a high definition map (turquoise) and the walkable area for the pedestrians (dark blue).

ACKNOWLEDGMENT

This publication was written in the framework of the Profilregion Mobilitätssysteme Karlsruhe, which is funded by the Ministry of Science and Research and the Arts and the Ministry of Economic Affairs, Labour and Housing in Baden-Württemberg and as a national High Performance Center by the Fraunhofer-Gesellschaft.

REFERENCES

- [1] B. W. Boehm, "Verifying and validating software requirements and design specifications," *IEEE Softw.*, vol. 1, no. 1, p. 75–88, Jan. 1984.
- [2] M. Quigley, K. Conley, B. P. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "Ros: an open-source robot operating system," in *ICRA Workshop on Open Source Software*, 2009.
- [3] M. R. Zofka, S. Ulbrich, D. Karl, T. Fleck, R. Kohlhaas, A. Rönnau, R. Dillmann, and J. M. Zöllner, "Traffic participants in the loop: A mixed reality-based interaction testbed for the verification and validation of autonomous vehicles," in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, Nov 2018, pp. 3583–3590.
- [4] C. Badue, R. Guidolini, R. V. Carneiro, P. Azevedo, V. B. Cardoso, A. Forechi, L. Jesus, R. Berriel, T. M. Paixão, F. Mutz *et al.*, "Self-driving cars: A survey," *Expert Systems with Applications*, 2020.
- [5] R. Matthaei and M. Maurer, "Autonomous driving—a top-down-approach," *at-Automatisierungstechnik*, vol. 63, no. 3, pp. 155–167, 2015.
- [6] J. Ziegler, P. Bender, M. Schreiber, H. Lategahn, T. Strauss, C. Stiller, T. Dang, U. Franke, N. Appenrodt, C. G. Keller, E. Kaus, R. G. Hertrich, C. Rabe, D. Pfeiffer, F. Lindner, F. Stein, F. Erbs, M. Enzweiler, C. Knöppel, J. Hipp, M. Haueis, M. Trepte, C. Brenk, A. Tamke, M. Ghanaat, M. Braun, A. Joos, H. Fritz, H. Mock, M. Hein, and E. Zeeb, "Making bertha drive—an autonomous journey on a historic route," *IEEE Intelligent Transportation Systems Magazine*, vol. 6, no. 2, pp. 8–20, Summer 2014.
- [7] O. S. Tas, N. O. Salscheider, F. Poggenhans, S. Wirges, C. Bandera, M. R. Zofka, T. Strauss, J. M. Zöllner, and C. Stiller, "Making bertha cooperate—team annieways entry to the 2016 grand cooperative driving challenge," *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, no. 4, pp. 1262–1276, April 2018.
- [8] O. S. Tas, F. Kuhnt, J. M. Zöllner, and C. Stiller, "Functional system architectures towards fully automated driving," in *2016 IEEE Intelligent Vehicles Symposium (IV)*, June 2016, pp. 304–309.
- [9] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proceedings of the 1st Annual Conference on Robot Learning*, 2017, pp. 1–16.
- [10] F. Rosique, P. J. Navarro, C. Fernández, and A. Padilla, "A systematic review of perception system and simulators for autonomous vehicles research," *Sensors*, vol. 19, no. 3, p. 648, 2019.
- [11] Y. Kang, H. Yin, and C. Berger, "Test your self-driving algorithm: An overview of publicly available driving datasets and virtual testing environments," *IEEE Transactions on Intelligent Vehicles*, vol. 4, pp. 171–185, 2019.
- [12] S. Hallerbach, Y. Xia, U. Eberle, and F. Köster, "Simulation-based identification of critical scenarios for cooperative and automated vehicles," *SAE Technical Papers*, pp. 93–106, Dezember 2018, SAE Intl. J CAV 1(2):2018.
- [13] M. R. Zofka, S. Klemm, F. Kuhnt, T. Schamm, and J. M. Zöllner, "Testing and validating high level components for automated driving: simulation framework for traffic scenarios," in *2016 IEEE Intelligent Vehicles Symposium (IV)*, June 2016, pp. 144–150.
- [14] P. A. Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wießner, "Microscopic traffic simulation using sumo," in *The 21st IEEE International Conference on Intelligent Transportation Systems*. IEEE, 2018.
- [15] N. Koenig and A. Howard, "Design and use paradigms for gazebo, an open-source multi-robot simulator," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, Sendai, Japan, Sep 2004, pp. 2149–2154.
- [16] A. Hussein, F. García, and C. Olaverri-Monreal, "Ros and unity based framework for intelligent vehicles control and simulation," in *2018 IEEE International Conference on Vehicular Electronics and Safety (ICVES)*, Sep. 2018, pp. 1–6.
- [17] C. Biurun-Quel, L. Serrano-Arriezu, and C. Olaverri-Monreal, "Microscopic driver-centric simulator: Linking unity3d and SUMO," in *Advances in Intelligent Systems and Computing*. Springer International Publishing, 2017, pp. 851–860.
- [18] L. Artal-Villa, A. Hussein, and C. Olaverri-Monreal, "Extension of the 3dcoautosim to simulate vehicle and pedestrian interaction based on sumo and unity 3d," in *2019 IEEE Intelligent Vehicles Symposium (IV)*, June 2019, pp. 885–890.
- [19] C. Sippl, B. Schwab, P. Kiehl, and A. Djanatliev, "Distributed real-time traffic simulation for autonomous vehicle testing in urban environments," in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, Nov 2018, pp. 2562–2567.
- [20] P. M. Kiehl, D. H. Biedermann, and A. Borrmann, "Momentumv2: A modular, extensible, and generic agent-based pedestrian behavior simulation framework," TU Munich, Tech. Rep., 2016.
- [21] A.-M. Hellmund, S. Wirges, Ö. S. Tas, C. Bandera, and N. O. Salscheider, "Robot operating system: A modular software framework for automated driving," *2016 IEEE 19th International Conference on Intelligent Transportation Systems (ITSC)*, pp. 1564–1570, 2016.
- [22] M. Naumann, F. Poggenhans, M. Lauer, and C. Stiller, "Coincar-sim: An open-source simulation framework for cooperatively interacting automobiles," in *2018 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2018, pp. 1–6.
- [23] M. L. Loper, "Introduction to distributed interactive simulation," in *Distributed Interactive Simulation Systems for Simulation and Training in the Aerospace Environment: A Critical Review*, T. L. Clarke, Ed., vol. 10280, International Society for Optics and Photonics. SPIE, 1995, pp. 9 – 21.
- [24] D. A. Fullford, "Distributed interactive simulation: Its past, present, and future," in *Proceedings of the 28th Conference on Winter Simulation*, ser. WSC '96. USA: IEEE Computer Society, 1996, p. 179–185.
- [25] G. Tibba, C. Malz, C. Stoermer, N. Nagarajan, L. Zhang, and S. Chakraborty, "Testing automotive embedded systems under x-in-the-loop setups," in *2016 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, Nov 2016, pp. 1–8.
- [26] M. Krammer, M. Benedikt, T. Blochwitz, K. Alekshin, N. Amringer, C. Kater, S. Materne, R. Ruvalcaba, K. Schuch, J. Zehetner *et al.*, "The distributed co-simulation protocol for the integration of real-time systems and simulation environments," in *Proceedings of the 50th Computer Simulation Conference*. Society for Computer Simulation International, 2018, p. 1.
- [27] N. Schröder, O. Lenord, and R. Lange, "Enhanced motion control of a self-driving vehicle using modelica, fmi and ros," 02 2019, pp. 441–450.
- [28] T. Hanke, N. Hirsenkorn, C. van Driesten, P. Garcia-Ramos, M. Schiementz, S. Schneider, and E. Biebl, "A generic interface for the environment perception of automated driving functions in virtual scenarios." 2017. [Online]. Available: <https://www.hot.ei.tum.de/forschung/automotive-veroeffentlichungen/>
- [29] S. Ulbrich, T. Menzel, A. Reschka, F. Schuldt, and M. Maurer, "Defining and substantiating the terms scene, situation, and scenario for automated driving," in *2015 IEEE 18th International Conference on Intelligent Transportation Systems*. IEEE, 2015, pp. 982–988.
- [30] S. Curtis, A. Best, and D. Manocha, "Menge: A modular framework for simulating crowd movement," *Collective Dynamics*, vol. 1, pp. 1–40, 2016.
- [31] A. Wegener, M. Piorkowski, M. Raya, H. Hellbrück, S. Fischer, and J.-P. Hubaux, "Traci: An interface for coupling road traffic and network simulators," *Proceedings of the 11th Communications and Networking Simulation Symposium, CNS'08*, 04 2008.
- [32] S. Curtis, "Pedestrian velocity obstacles: Pedestrian simulation through reasoning in velocity space," Ph.D. dissertation, University of North Carolina at Chapel Hill, 2013.
- [33] C. Crick, G. Jay, S. Osentoski, B. Pitzer, and O. Jenkins, *Rosbridge: ROS for Non-ROS Users*, 01 2017, vol. 100, pp. 493–504.
- [34] P. Mania and M. Beetz, "A framework for self-training perceptual agents in simulated photorealistic environments," in *International Conference on Robotics and Automation*, Montreal, Canada, 2019.
- [35] L.-A. Mathis, F. Diederichs, H. Widroither, D. Ruscio, L. Napolitano, M. R. Zofka, A. Viehl, P. Fröhlich, J. Friedrich, A. Lindström, B. Thorslund, A. Litke, N. Papadakis, N. Bakalos, U. Hernandez-Jayo, S. Espie, V. Cavallo, M. Panou, E. Gaitanidou, and E. Bekiaris, "Creating informed public acceptance by a user-centered human-machine interface for all automated transport modes," in *Proceedings of 8th Transport Research Arena TRA 2020* :, 2020, pp. 9–.